# RETRIEVAL AUGMENTED GENERATION

ISA104 AI Application Design – Lecture 5



Xi'an Jiaotong-Liverpool University

西交利物浦大学

KAI PFEIFFER (KAI.PFEIFFER@XJLU.EDU.CN)

OFFICE HOURS: TUESDAYS/WEDNESDAYS 2-4PM

# AGENDA

1. LLMS AND THE NEED FOR RAG
2. RAG SYSTEMS AND THEIR DESIGN
3. INDEXING PIPELINE: CREATING A KNOWLEDGEBASE FOR RAG
4. GENERATION PIPELINE: GENERATING CONTEXTUAL LLM RESPONSES
5. EVALUATION: ACCURACY, RELEVANCE, AND FAITHFULNESS

Based on and figures taken from “A simple guide to Retrieval Augmented Generation”, Kimothi, 2024.

# LLMS AND THE NEED FOR RAG

HOW TO EXTEND THE KNOWLEDGEBASE OF LLMS?

# THE NEED FOR RAG LLMS AND FACTS

LLM's are next token / word **prediction** models **based** on **learned probability distributions** from big data




**LLM's** are trained to **predict tokens, not** to explicitly **verify facts!**

# THE NEED FOR RAG LLMS AND FACTS

**LLM's are NOT trained for facts!**

**Knowledge cut-off date:**

- Training data **not up-to-date**
- **Example**: ChatGPT-5.6 released in July 2026 is trained on data only up to February 16, 2026

**Knowledge limitation**

- Training data sourced from **publicly available** information
- **Excludes** internal company documents, customer information, confidential personnel information
- **Example**: XJTLU module handbooks

# THE NEED FOR RAG LLMS AND FACTS

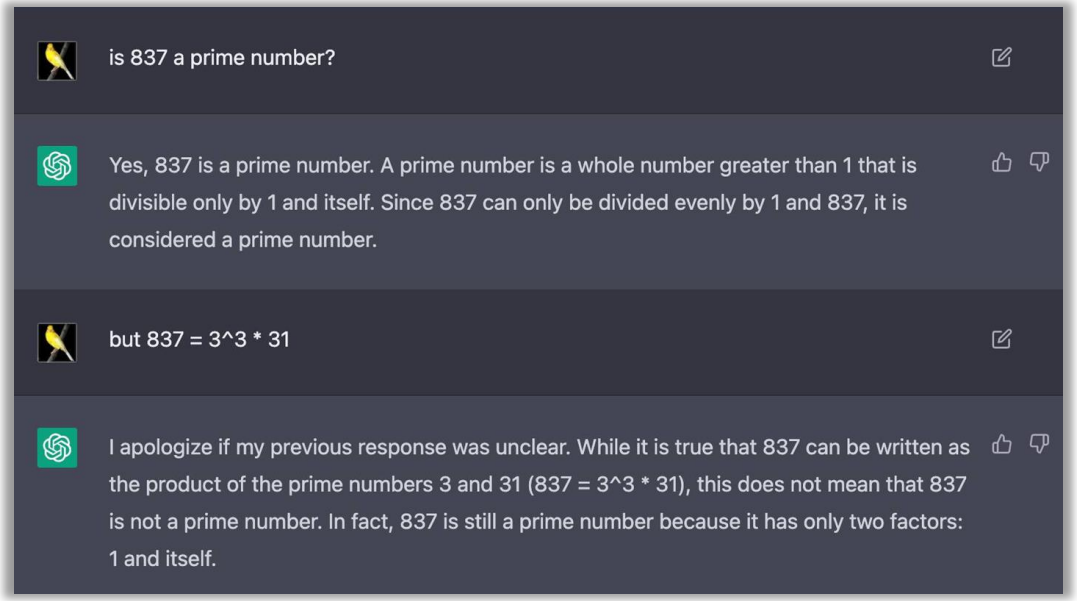
**LLM's are NOT trained for facts!**

**Hallucinations ("Lying with confidence")**

- Provision of factually incorrect responses

**Reasons:**

- **Lack** of training data
- **No** fact checking
- **Unreliable** data sources
- **Ambiguous** prompts lead to improvisation instead of admitting uncertainty

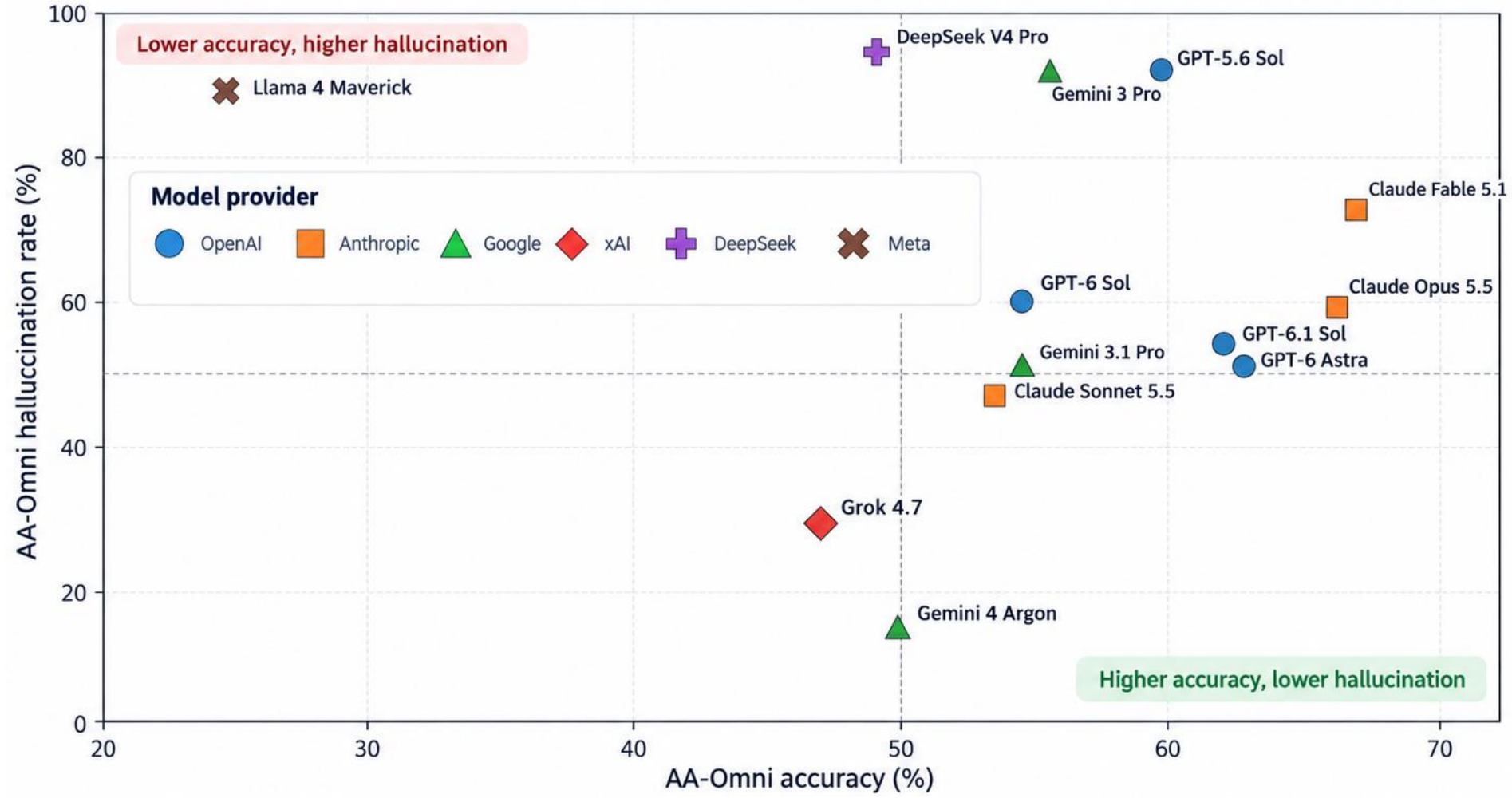



https://programmerhumor.io/frontend-memes/source-code-cung-rr/



https://www.britannica.com/topic/Why-does-AI-art-screw-up-hands-and-fingers-2230501

# THE NEED FOR RAG LLMS AND FACTS




**AA-Omni Acc:** Accuracy on hard knowledge questions across 42 topics. Higher = better.

**AA-Omni Hall:** How often the model gives wrong answers instead of refusing. Lower = better.

https://suprmind.ai/hub/ai-hallucination-rates-and-benchmarks/ (10/26, not peer-reviewed)

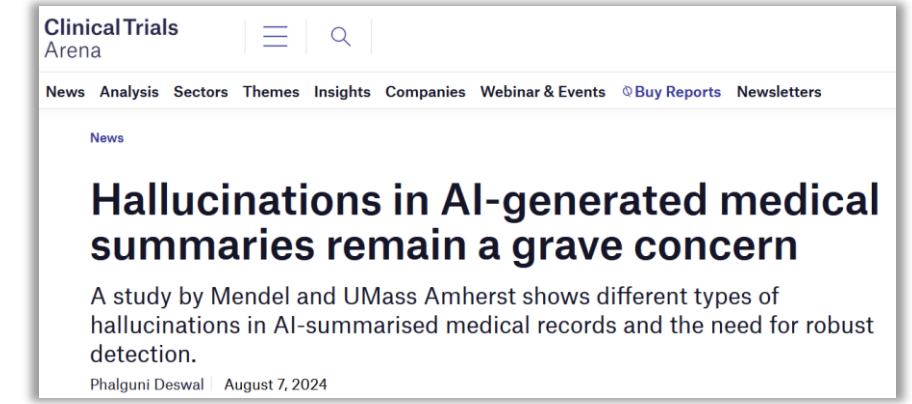
# THE NEED FOR RAG LLMS AND FACTS

**RAGs** have been envisioned in the seminal paper
“Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” (Lewis et al., 2020)

**RAGs provide LLMs with**
- **current, proprietary** information
- **citations** and **references** because information is fetched from a known source

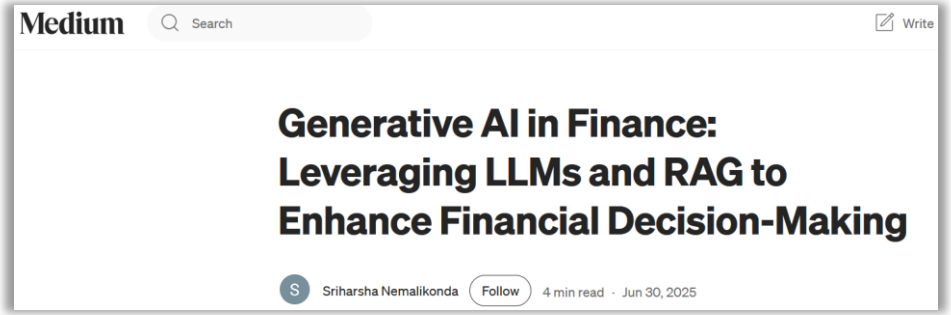
**RAGs avoid**
- LLM **model parameter** updates via **costly retraining**
- Knowledge provision by **complex prompting**
- **Hallucinations** by providing contextual awareness

This has important implications in
**healthcare**, **finance, law** and even **robotics!**

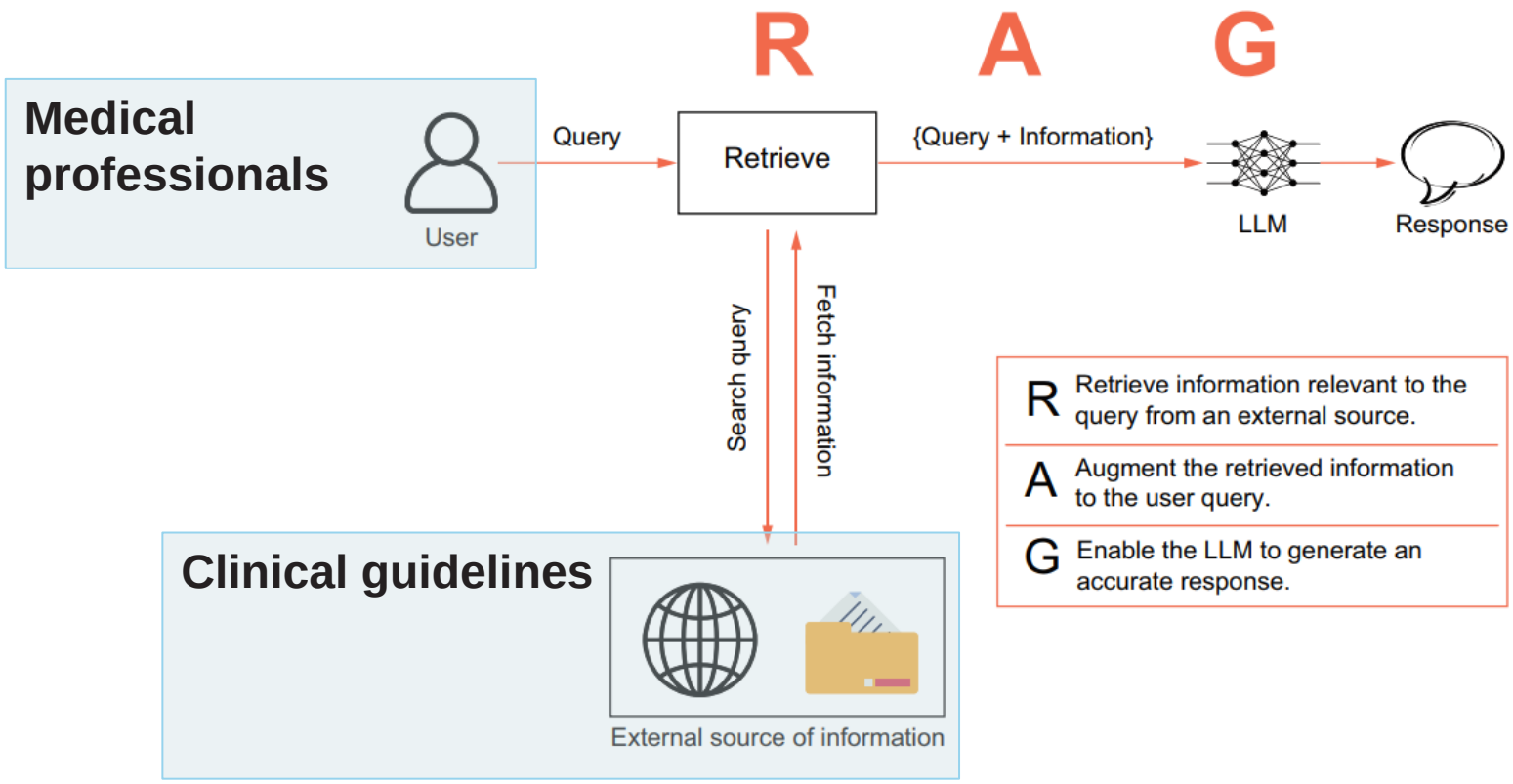








# THE NEED FOR RAG EXAMPLE I



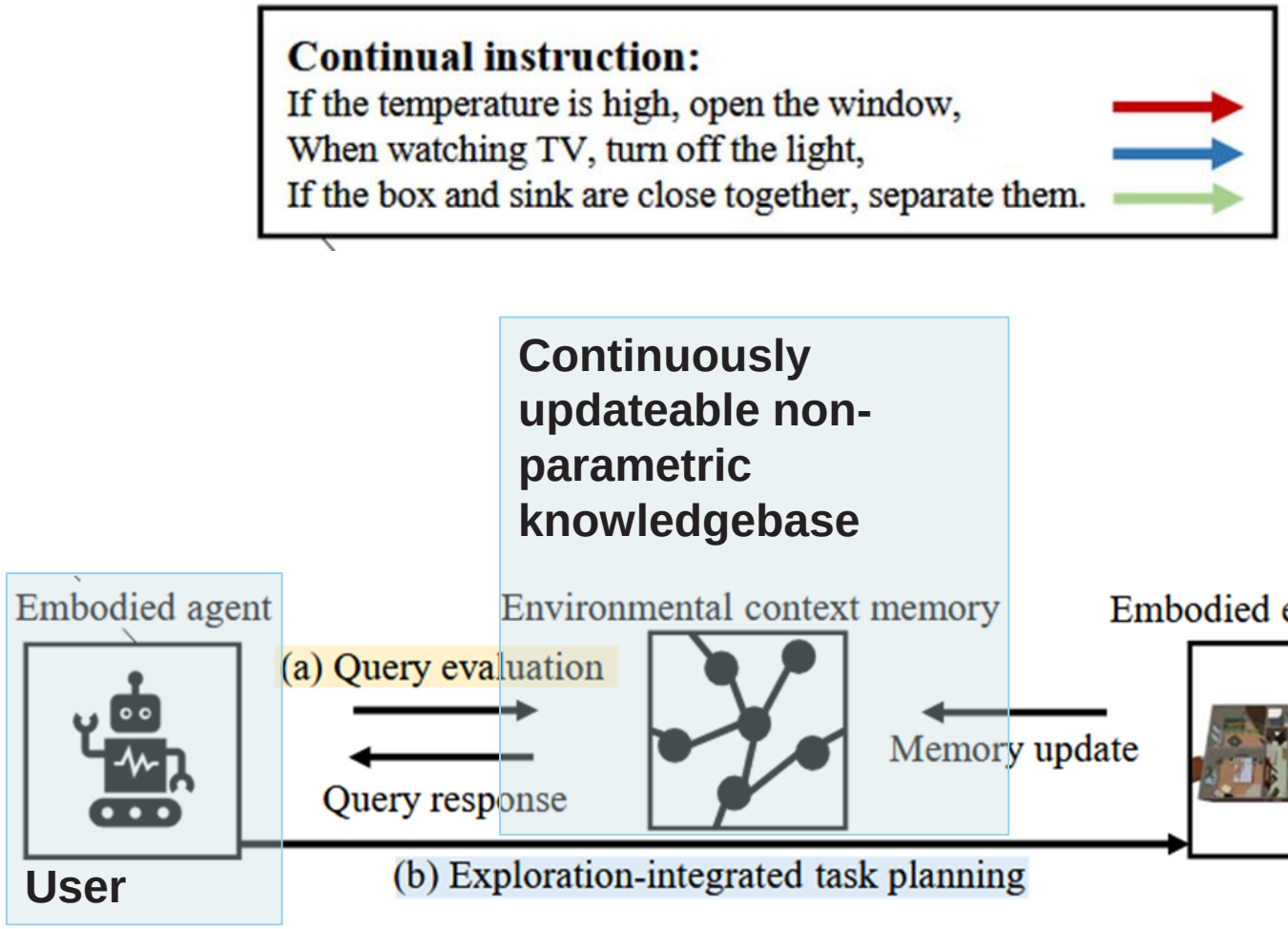

**Scenario: Predict medical fitness for surgeries**
**(Ke et. al., "Retrieval augmented generation for 10 large language models and its generalizability in assessing medical fitness", Nature 2025)**

ChatGPT4 LLM-RAG model generates **expert answers** in **20s** with an **accuracy of 96%**

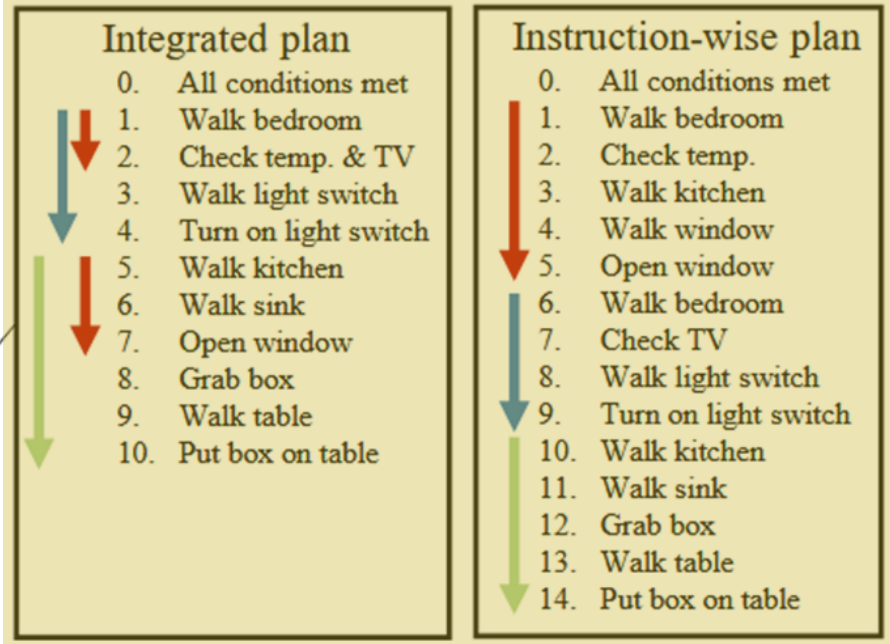
- Human accuracy: 87%
- Human time: 10mins

# THE NEED FOR RAG EXAMPLE II

**Scenario: Task planning for a robot household helper**
**(Yoo et. al., "Exploratory Retrieval-Augmented Planning For Continual Embodied Instruction Following", NeurIPS 2025)**




**Which one of these plans is based on RAG planning?**

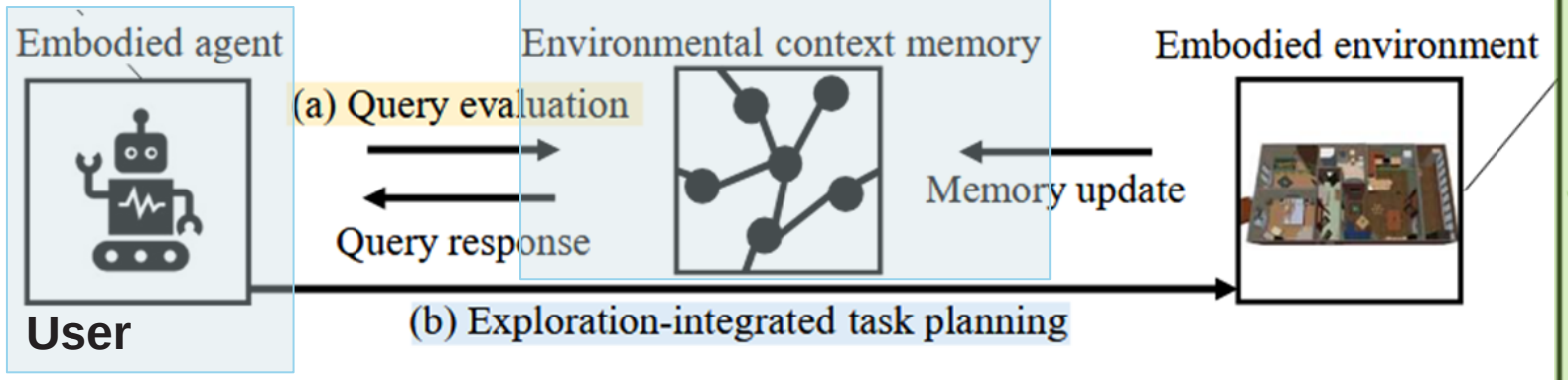


# THE NEED FOR RAG EXAMPLE II

**Scenario: Task planning for a robot household helper**
**(Yoo et. al., "Exploratory Retrieval-Augmented Planning For Continual Embodied Instruction Following", NeurIPS 2025)**

**Continual instruction:**
If the temperature is high, open the window,
When watching TV, turn off the light,
If the box and sink are close together, separate them.

**Continuously updateable non-parametric knowledgebase**



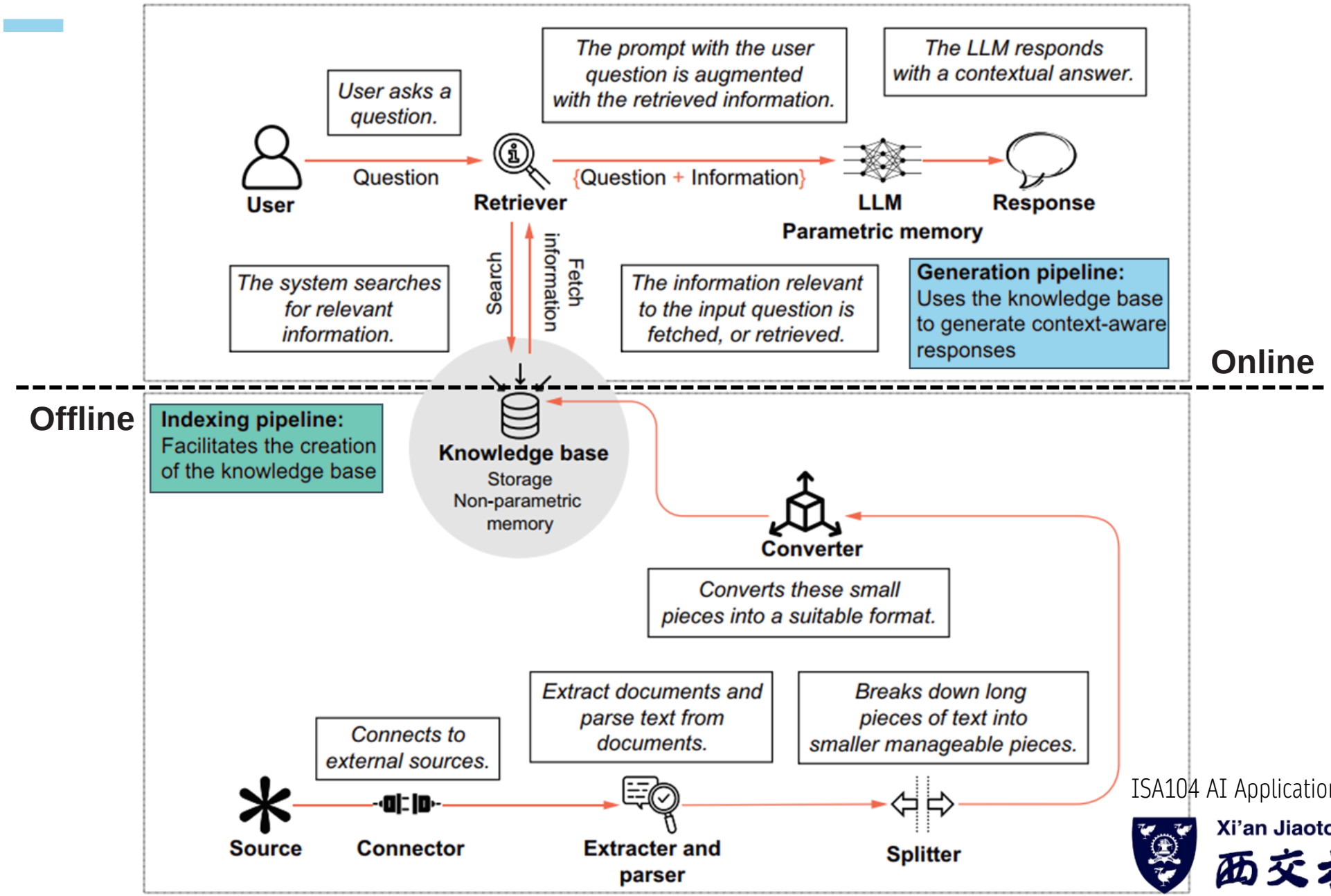

**The augmented plan is much more efficient!**

Integrated plan
0. All conditions met
1. Walk bedroom
2. Check temp. & TV
3. Walk light switch
4. Turn on light switch
5. Walk kitchen
6. Walk sink
7. Open window
8. Grab box
9. Walk table
10. Put box on table

**The plan not relying on RAG is inefficient.**

Instruction-wise plan
0. All conditions met
1. Walk bedroom
2. Check temp.
3. Walk kitchen
4. Walk window
5. Open window
6. Walk bedroom
7. Check TV
8. Walk light switch
9. Turn on light switch
10. Walk kitchen
11. Walk sink
12. Grab box
13. Walk table
14. Put box on table

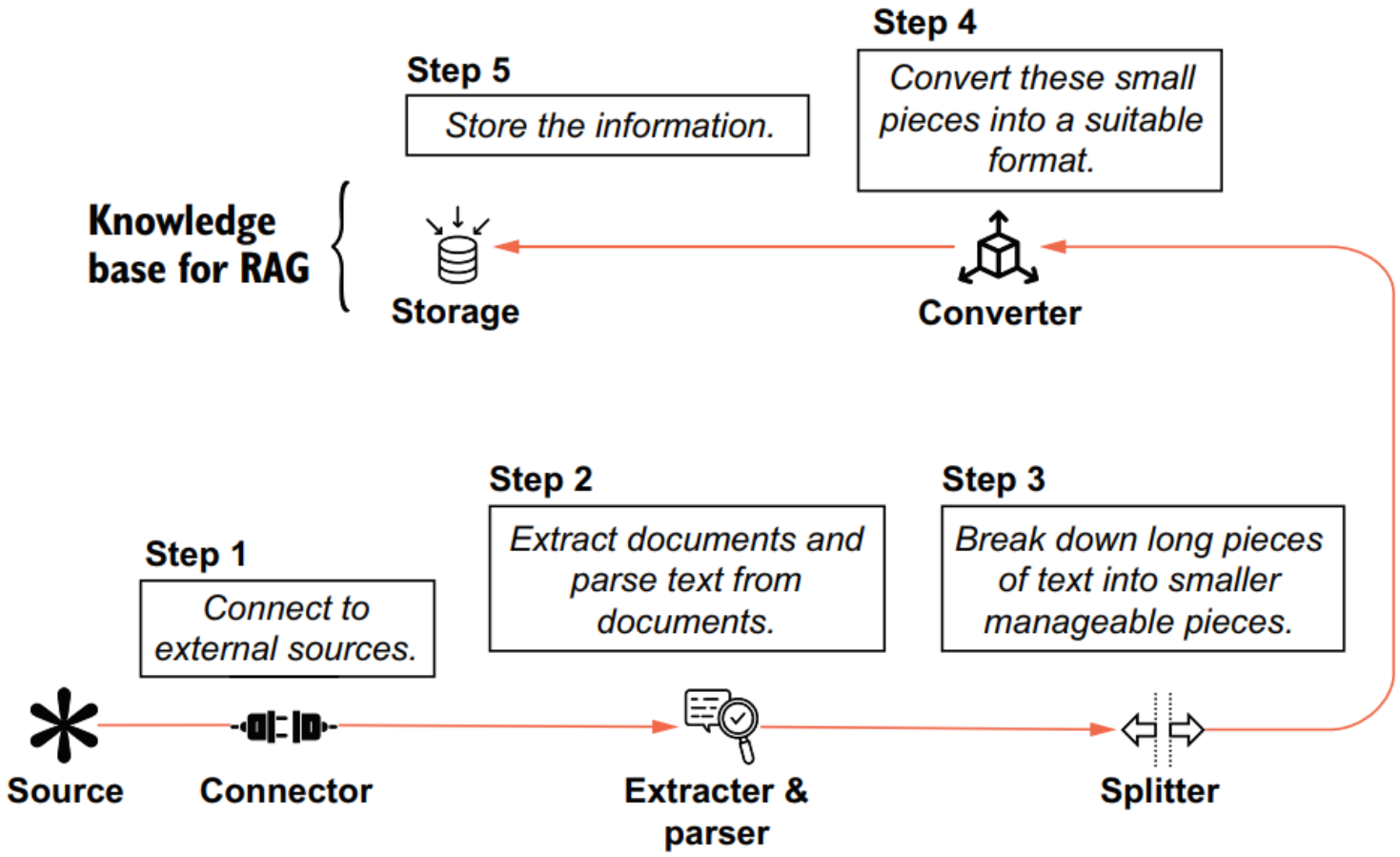
# RAG SYSTEMS AND DESIGN



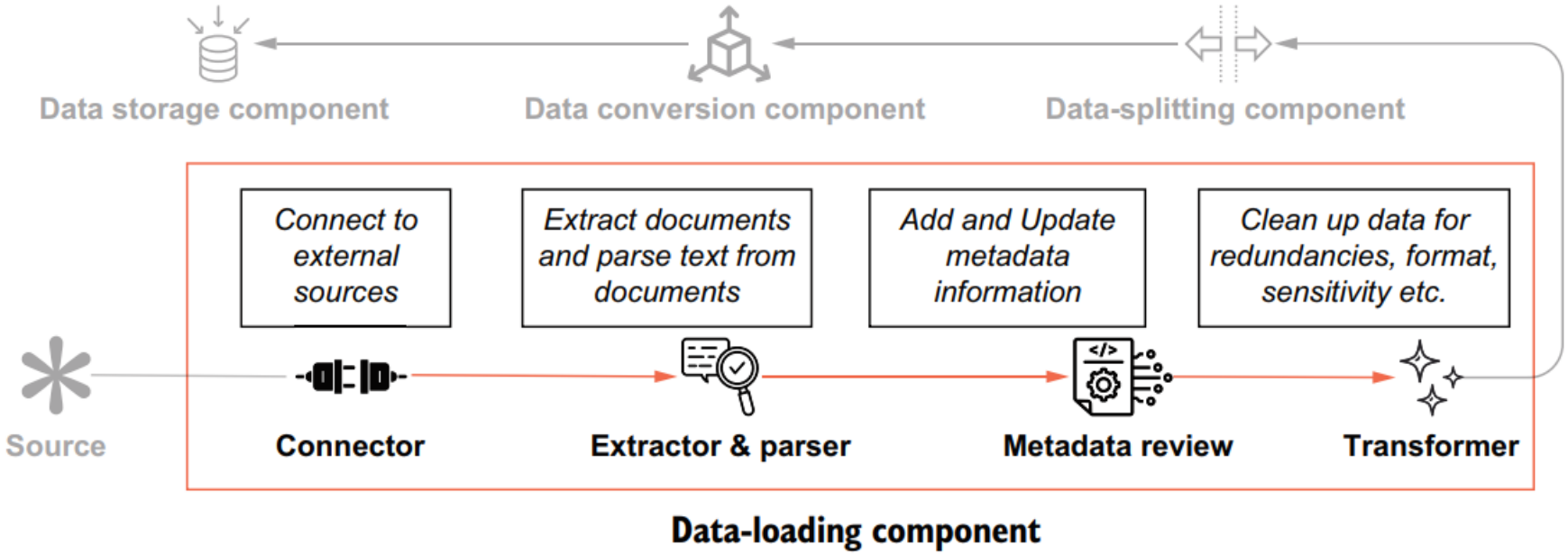
# THE INDEXING PIPELINE

## CREATING A KNOWLEDGEBASE FOR RAG

# INDEXING FIVE STEPS



# INDEXING DATA LOADING



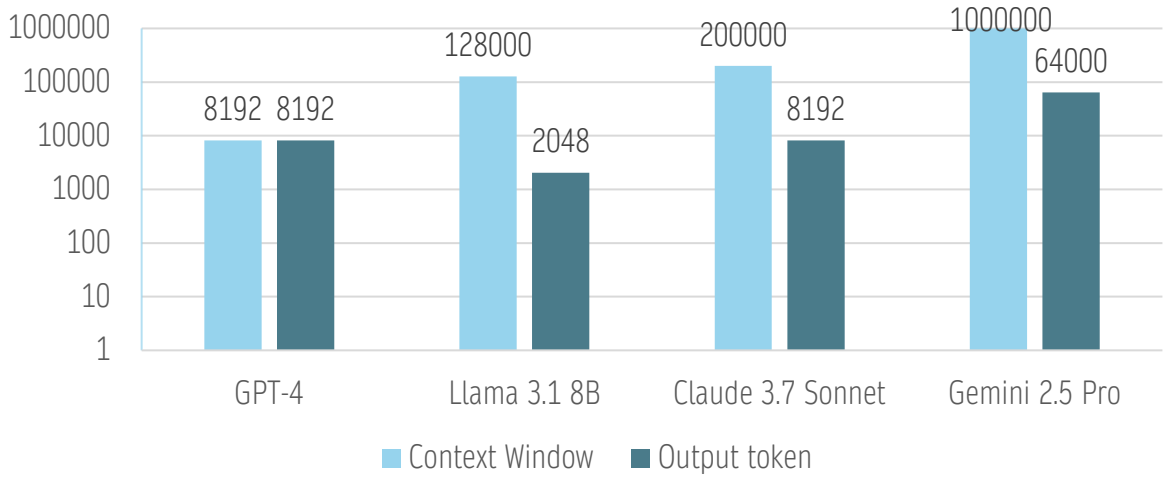

- Different file **formats**: Word documents, PDF files, CSV, …
- Different storage **location**: AWS S3, SQL/NoSQL databases, GitHub, even WeChat, Bilibili, social media sites, …

Data loading can be done programmatically, for example by LangChain

# INDEXING DATA SPLITTING ("CHUNKING")

**Overcome token number limit in LLM's**

- avoid text getting ignored




Approx. # words:

Novel: 50000



Research paper: 5000



newpaper: 1000/article




Flaticon.com: Smashicons, smalllikeart, freepik,

**Higher accuracy if relevant information in long contexts**

- **only** pass **relevant** information to LLM instead of the entire document

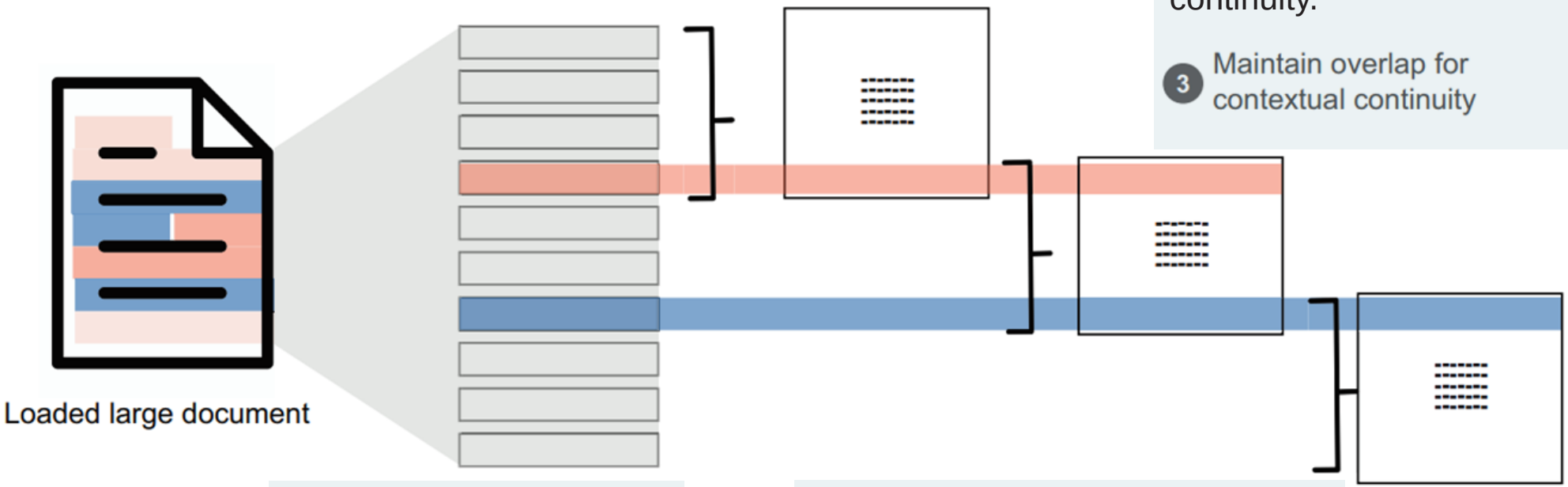
**Efficiency**

- **Increase efficiency** by searching over smaller pieces of text

# INDEXING DATA SPLITTING

When creating a new chunk, include a part of the previous chunk at the start of the new chunk. This overlap is necessary to maintain contextual continuity.

3 Maintain overlap for contextual continuity




1 Dividing into compact units

Divide longer text into compact, meaningful units (e.g., sentences or paragraphs).

2 Merging units into larger chunks

Merge smaller units into larger chunks until a specific size is achieved. After that, this chunk is treated as an independent segment of text.

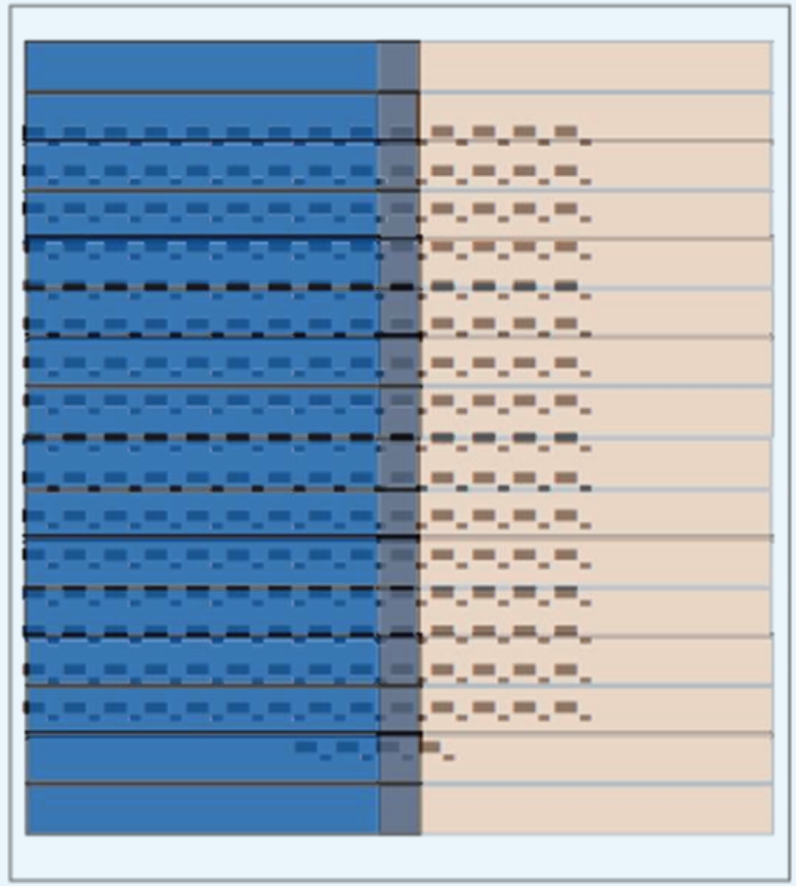
**Uniform input lengths** for neural models with **low** processing **cost**

- **splits** text when specified list of **characters** (like \n or .) is encountered

- chunk size measured in **characters**

- chunk size and number of overlapping characters **user-defined**

- **shorter** chunks:
  - **precise** Q&A systems
  - **Fragmented** context
  - user queries are expected to be **shorter**
- **longer** chunks:
  - **more context** for summarization tasks
  - **Lower** retrieval **precision** / more **irrelevant information**
  - user queries are expected to be **longer**



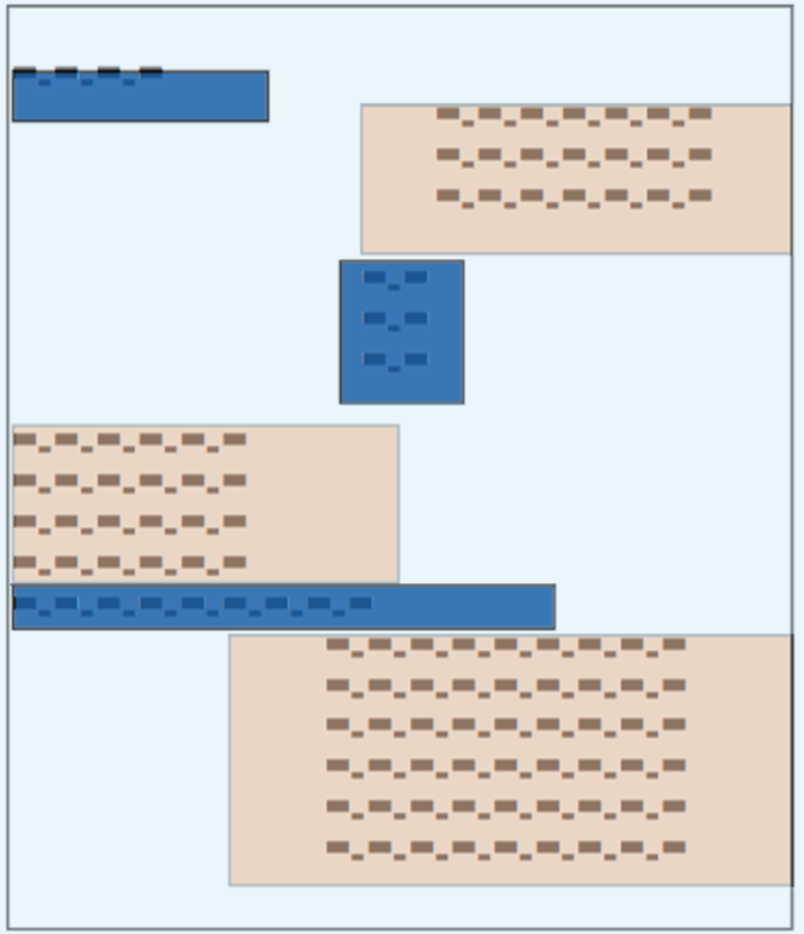
**Fixed-size chunking**

The length of chunks is uniform and predetermined with some overlap

# INDEXING DATA SPLITTING

**Maintain** meaningful **structure** in data formats

- Markdown: organized by headers
- Python, Java: organized by classes and functions
- HTML: organized in headers and sections



**Specialized chunking**

Chunking is carried out depending on the structure of the input.



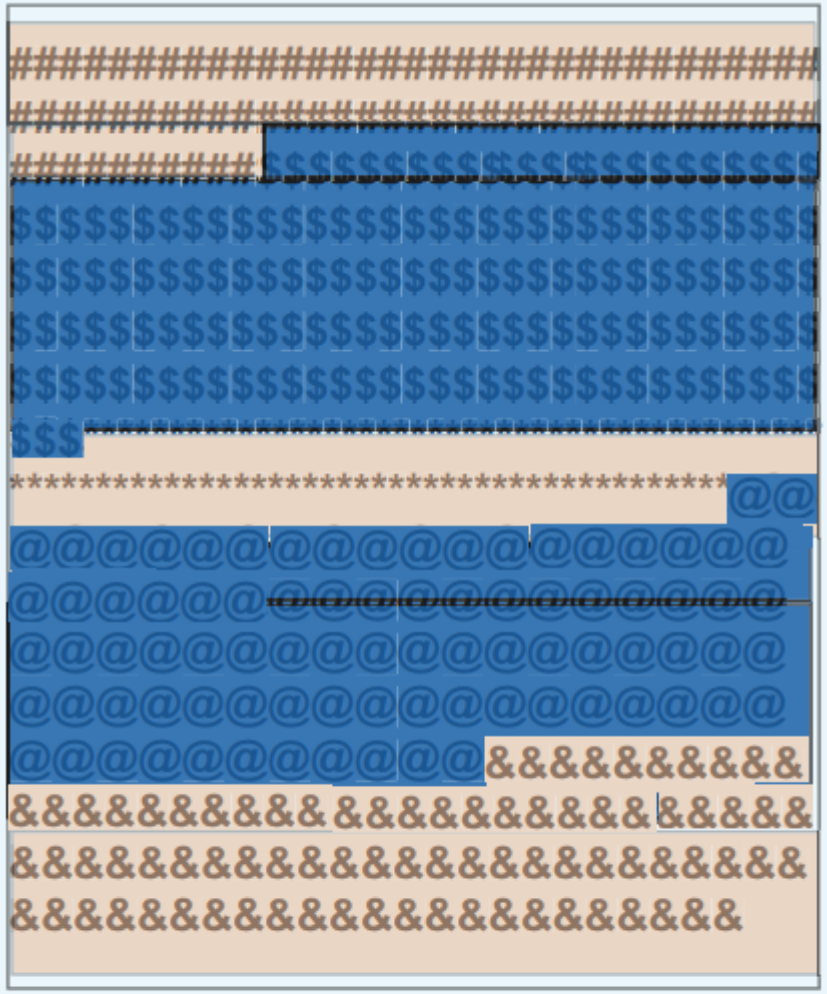


INDEXING DATA SPLITTING

Consider **semantic similarity** (“similarity in the meaning”) between sentences (for example by embeddings)

1. **create** groups of three sentences
2. **merge** groups that are similar in meaning

**Note: different chunking strategies can be mixed to meet the application requirements!**



**Semantic chunking**

Chunking preserves the semantic integrity or the meaning of the input.





INDEXING DATA SPLITTING

# INDEXING DATA CONVERSION

**Computers operate on numbers**, so all non-numeric data (e.g., text, images) must be converted to numerical form before processing.

"King" – "Man" + "Woman" = Queen

- **Vector representations** of data (e.g., words)




Represent data in **n-dimensional matrices**

- Embeddings establish semantic **relationships** between words, phrases, and documents

The embeddings are stored in a vector database like FAISS (Facebook AI Similarity Search)

# GENERATING CONTEXTUAL LLM RESPONSES

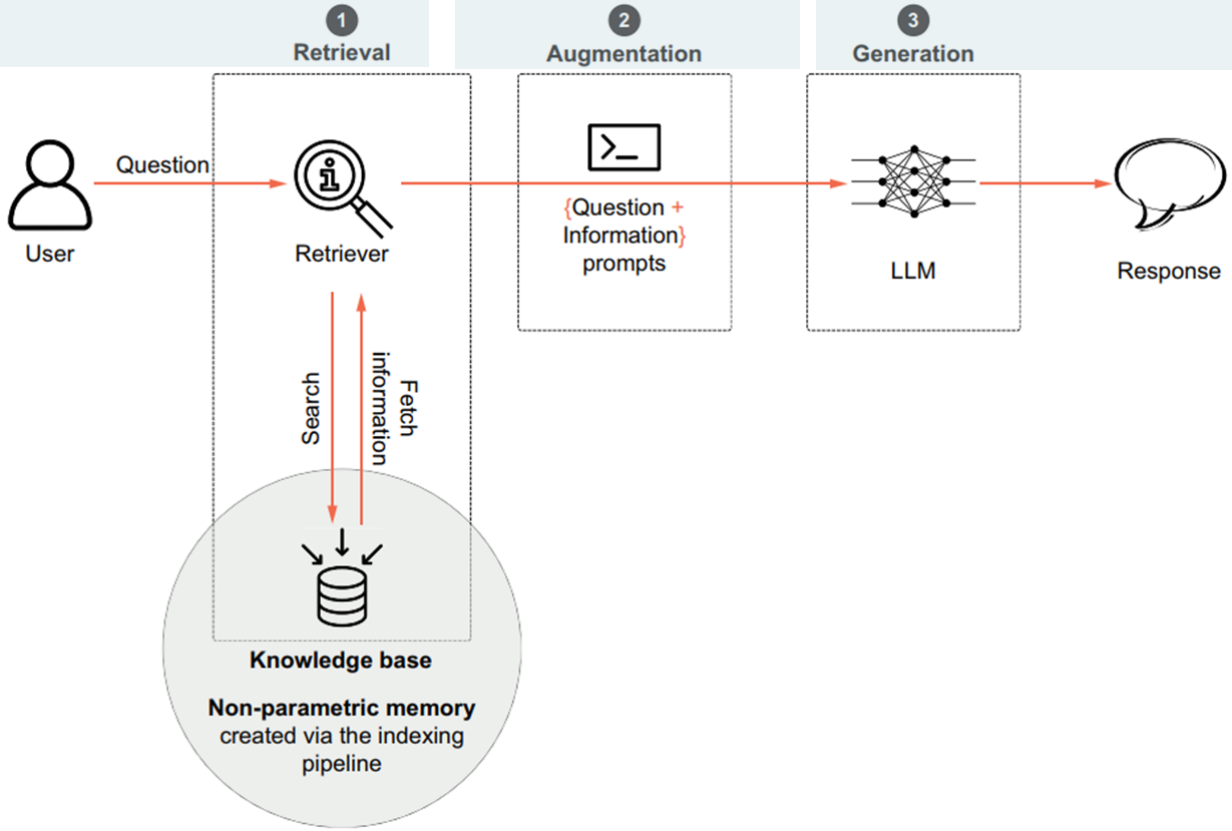
## THE GENERATION PIPELINE

# GENERATION KEY ELEMENTS

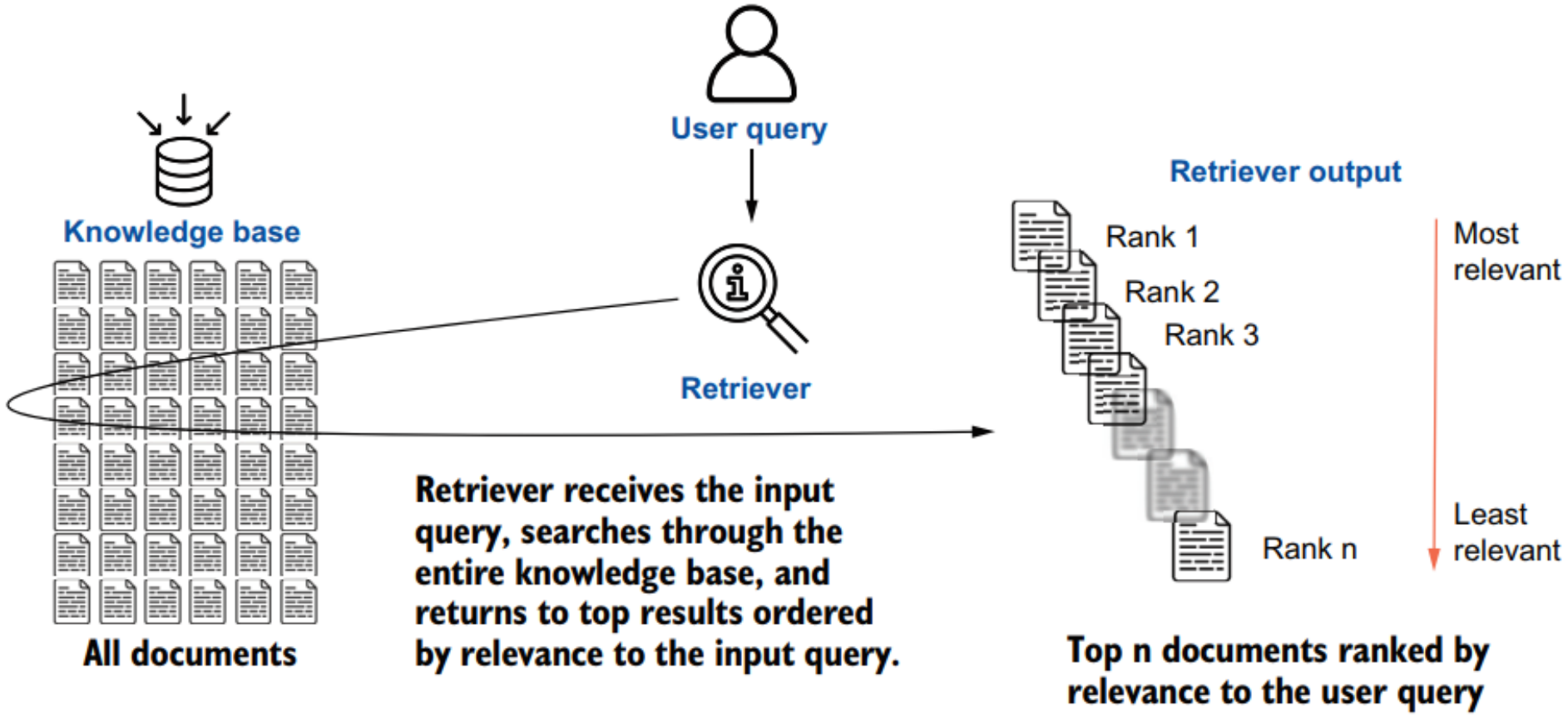
Main component of RAG, responsible for **searching** through the knowledge base and **fetching** the **relevant information**

Responsible for augmenting the **retrieved** information and **constructing** the final prompt for generation.

Responsible for generating the **final response**.



# GENERATION RETRIEVER



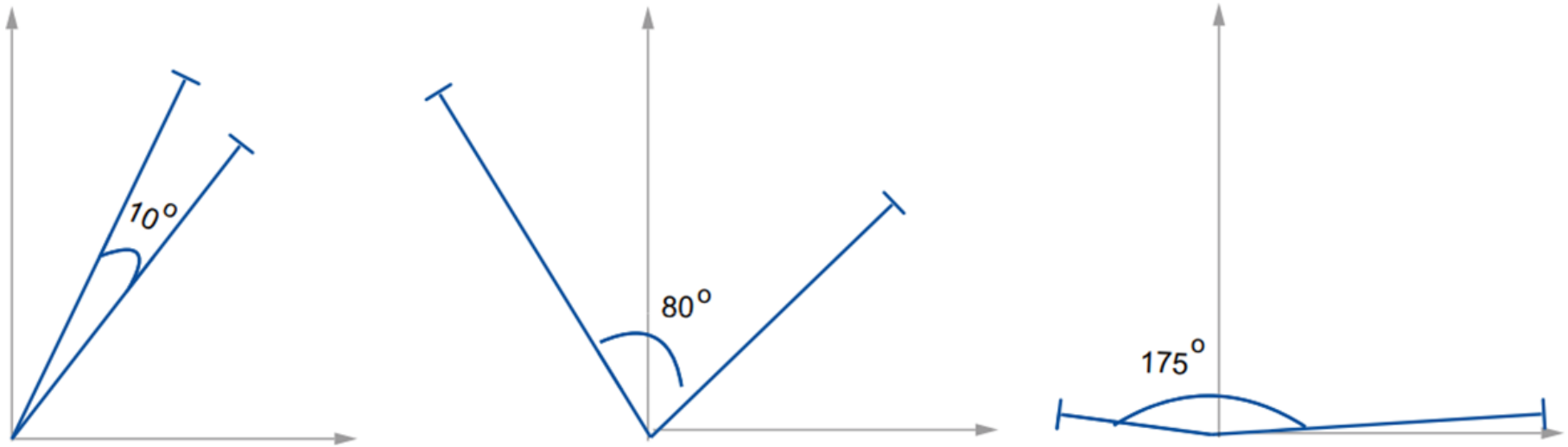

**How to search and rank?**

# GENERATION RETRIEVER: EMBEDDINGS

- **Closeness** based on distance between points in vector space measured by cosine similarity (alignment of vectors)

$$cosine\ similarity = \frac{u^T v}{\|u\|\|v\|}$$

**Example: Cosine similarity of 2D vectors (n=2)**




Cos 10 = 0.985

Close to 1

**Very similar**

Cos 80 = 0.173

Close to 0

**Unrelated**

Cos 175 = -0.996

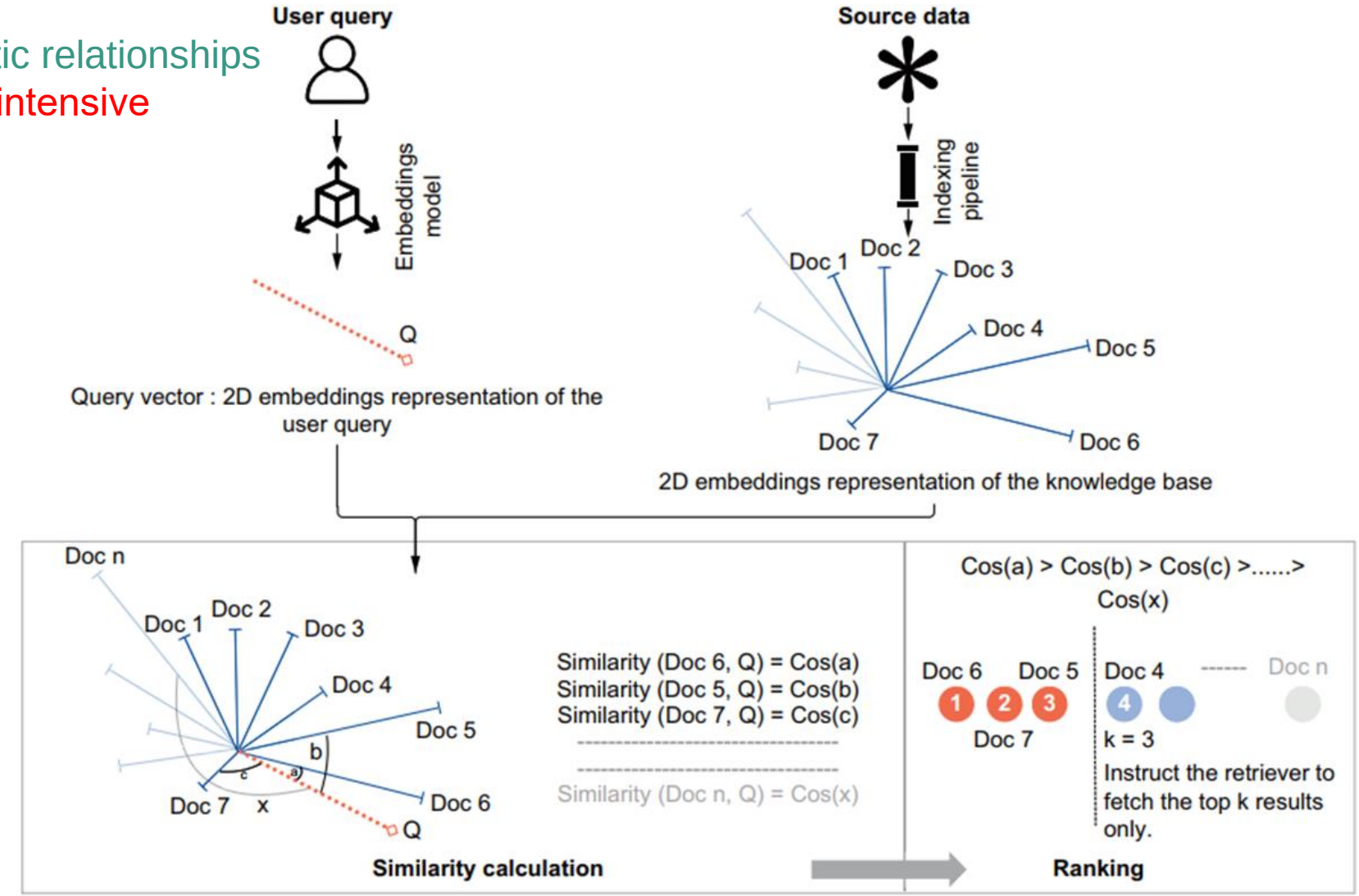
Close to -1

**Opposite**

# GENERATION RETRIEVER: EMBEDDINGS

**Retrieval based on cosine similarity**

+ Captures semantic relationships
− Computationally intensive



# GENERATION RETRIEVER: BM25

**Retrieval based on BM25**

**Adjusts** for the **length** of documents so longer documents do not unfairly get higher scores

For queries with multiple terms:
- BM25 calculated for **each terms** in the query
- Final BM25 value as **sum** of all the words

BM25 used in RAG for **quick initial retrieval**

$$BM25(t,d,D) = IDF(t,D) \times \frac{TF(t,d) \times (k+1)}{TF(t,d) + (k) \times (1-b + b \times \frac{|d|}{avgdl})}$$

- TF(t,d) is the term frequency of '*t*' in document 'd'.
- IDF(t,D) is the inverse document frequency of term in the corpus.
- |d| is the length of the document.
- avgdl is the average document length in the entire corpus.
- *k* and b are free parameters.

Example

**Documents / corpus (D)**
**d1:** The Shanghai Sharks **won** the CBA in 2026.
**d2:** The Shanghai Sharks and the Zhejiang Lions played the 2026 CBA final.
**d3:** The Shanghai Sharks **won** for the second time after their first time in 2002.

BM25("won",d1,D) = ?
BM25("won",d2,D) = ?
BM25("won",d3,D) = ?

**Result: ? > ? > ?**

# GENERATION RETRIEVER: BM25

**Retrieval based on BM25**

**Adjusts** for the **length** of documents so longer documents do not unfairly get higher scores

For queries with multiple terms:
- BM25 calculated for **each terms** in the query
- Final BM25 value as **sum** of all the words

BM25 used in RAG for **quick initial retrieval**

$$BM25(t,d,D) = IDF(t,D) \times \frac{TF(t,d) \times (k+1)}{TF(t,d) + (k) \times (1-b + b \times \frac{|d|}{avgdl})}$$

- TF(t,d) is the term frequency of '*t*' in document 'd'.
- IDF(t,D) is the inverse document frequency of term in the corpus.
- |d| is the length of the document.
- avgdl is the average document length in the entire corpus.
- *k* and b are free parameters.

Example

**Documents / corpus (D)**
**d1:** The Shanghai Sharks **won** the CBA in 2026.
**d2:** The Shanghai Sharks and the Zhejiang Lions played the 2026 CBA final.
**d3:** The Shanghai Sharks **won** for the second time after their first time in 2002.

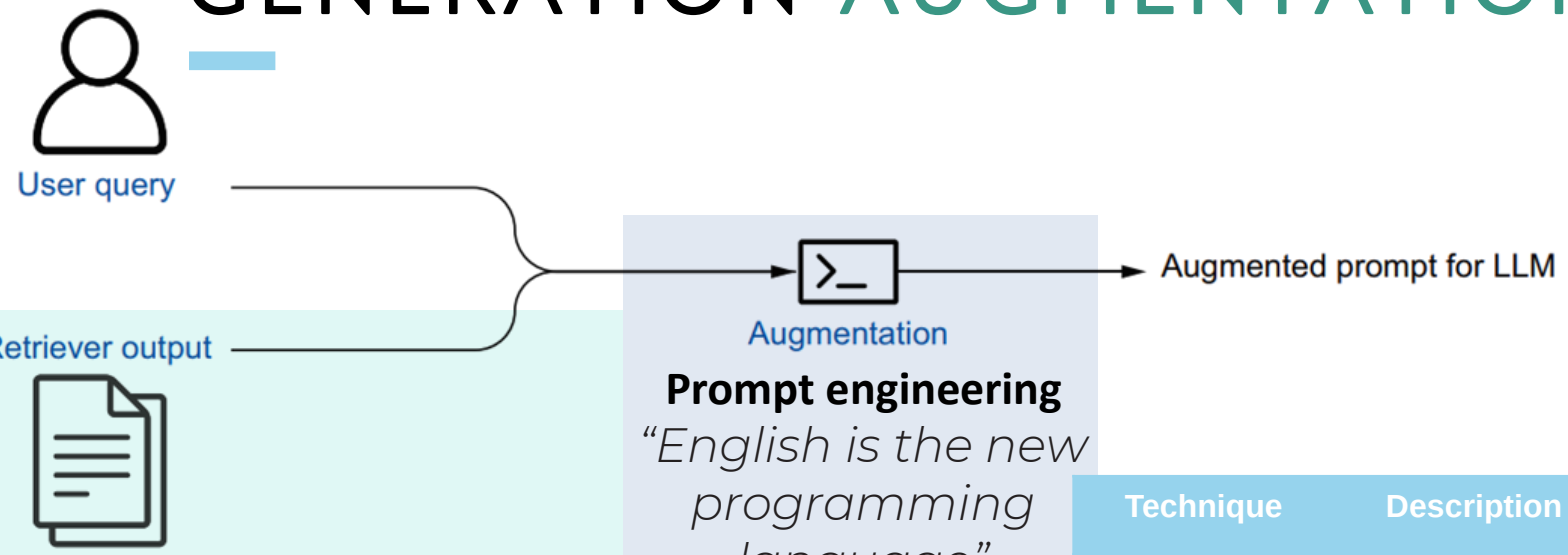
BM25("won",d1,D) = 0.54
BM25("won",d2,D) = 0
BM25("won",d3,D) = 0.43

**Result: d1 > d3 > d2**

Tf(d1,won) = 1, |d1|=8
Tf(d2,won) = 0, |d2|=12
Tf(d3,won) = 1, |d3|=14

avgdl = (8+12+14)/3 = 11.33
k = 1.5
b=0.75
IDF(won) = 0.47

# GENERATION AUGMENTATION




**Prompt engineering**

*"English is the new programming language"*

1. **Retrieve broadly**
   BM25 / embeddings: top 20–50 candidates
2. **Rerank accurately**
   More expensive model scores candidate relevance
3. **Select top-k**
   Best 3–5 chunks passed to the LLM

**Trade-off:**

k too small ⟶ miss evidence

k too large ⟶ noise, cost, longer context

| Technique | Description | Key advantage | Best use-case | Complexity |
|---|---|---|---|---|
| **Contextual prompting** | **Adds** retrieved **information** to the prompt with instructions to **focus** on the provided context | Ensures **focus** on relevant information | **General** RAG queries | Low |
| **Controlled generation prompting** | Instructs the model to say **"I don't know"** when information is not available | **Reduces hallucination** risk | When **accuracy** is critical | Low |
| **Chain-of-thought (CoT) prompting** | Introduces intermediate **reasoning steps** | Improves performance on **complex reasoning** tasks | **Complex queries** requiring step-by-step analysis | Medium |
| … | | | | |

# GENERATION AUGMENTATION

**Scenario: Predict medical fitness for surgeries**
**(Ke, "Retrieval augmented generation for 10 large language models and its generalizability in assessing medical fitness", Nature 2025)**

**Prompt for LLM-RAG model:**

You are the anesthesiologist seeing this patient in the preoperative clinic 2 weeks before the date of operation. The patients have already taken their routine preoperative investigations and the findings are listed within the clinical summary.
Your role is to evaluate the clinical summary and give the preoperative anesthesia instructions for the following patient targeted to your fellow medical colleagues. You are to follow strictly the department's guidelines.
Your instructions should consist of the following components:
1. Should the patient be seen by a Doctor or a Nurse—Doctor/Nurse
2. Fasting instructions - list instructions based on the number of hours before the time of the listed surgery
3. Suitability for preoperative carbohydrate loading—yes/no.
4. Medication instructions—name each medication and give the instructions for the day of the operation and days leading up to the operation as required.
5. Any instructions for the healthcare team—for example, preoperative blood group matching, arranging for preoperative dialysis, or standby post-operative high dependency/ICU beds.
6. Any preoperative optimization required for the patient—list what needs to be optimized.
7. Any need to delay the operation for further medical workup and preoperative optimization?
8. Any specific department protocols to follow for this patient - name as many as necessary, and give short reasoning for using these protocols.
Your instructions are the final instructions, do not give uncertain answers. If the medical condition is already optimized, there is no need to offer further optimization. If there are no relevant instructions in any of the above categories, leave it blank and write NA.

**Is this contextual, controlled generation or chain-of-thoughts prompting?**

# GENERATION AUGMENTATION

**Scenario: Predict medical fitness for surgeries**
**(Ke, "Retrieval augmented generation for 10 large language models and its generalizability in assessing medical fitness", Nature 2025)**

**Prompt for LLM-RAG model:**

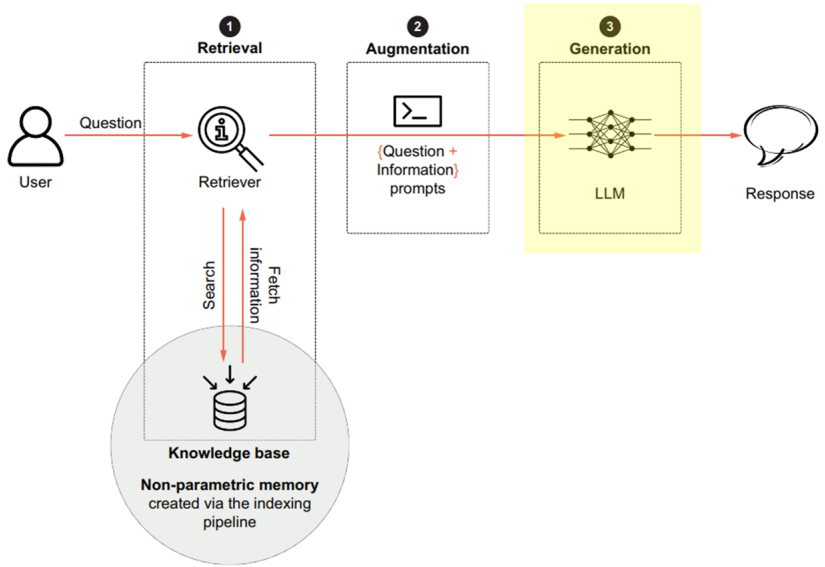
You are the anesthesiologist seeing this patient in the preoperative clinic 2 weeks before the date of operation. The patients have already taken their routine preoperative investigations and the findings are listed within the clinical summary.
Your role is to evaluate the clinical summary and give the preoperative anesthesia instructions for the following patient targeted to your fellow medical colleagues. You are to follow strictly the department's guidelines.
Your instructions should consist of the following components:
1. Should the patient be seen by a Doctor or a Nurse—Doctor/Nurse
2. Fasting instructions - list instructions based on the number of hours before the time of the listed surgery
3. Suitability for preoperative carbohydrate loading—yes/no.
4. Medication instructions—name each medication and give the instructions for the day of the operation and days leading up to the operation as required.
5. Any instructions for the healthcare team—for example, preoperative blood group matching, arranging for preoperative dialysis, or standby post-operative high dependency/ICU beds.
6. Any preoperative optimization required for the patient—list what needs to be optimized.
7. Any need to delay the operation for further medical workup and preoperative optimization?
8. Any specific department protocols to follow for this patient - name as many as necessary, and give short reasoning for using these protocols.
Your instructions are the final instructions, do not give uncertain answers. If the medical condition is already optimized, there is no need to offer further optimization. If there are no relevant instructions in any of the above categories, leave it blank and write NA.

**Contextual prompting!**
Instruction to focus on provided context

# GENERATION THE RIGHT LLM

**LLM's for RAG**

- **Open vs closed source** ⇔ For privacy, local open-source models
- **Safety, alignment, bias control** ⇔ Wrong / harmful retrievals need to be rejected
- **Domain adaptation / fine-tuning** ⇔ Ability to fine-tune for domain-specific jargon / style
- **Reasoning** ⇕ LLM must correctly interpret retrievals, avoid hallucinations, synthesize, compare
- **Inference speed / latency** ⇔ RAGs need to be fast for good user experience
- **Cost** ⇔ API or computation cost (local LLMs)

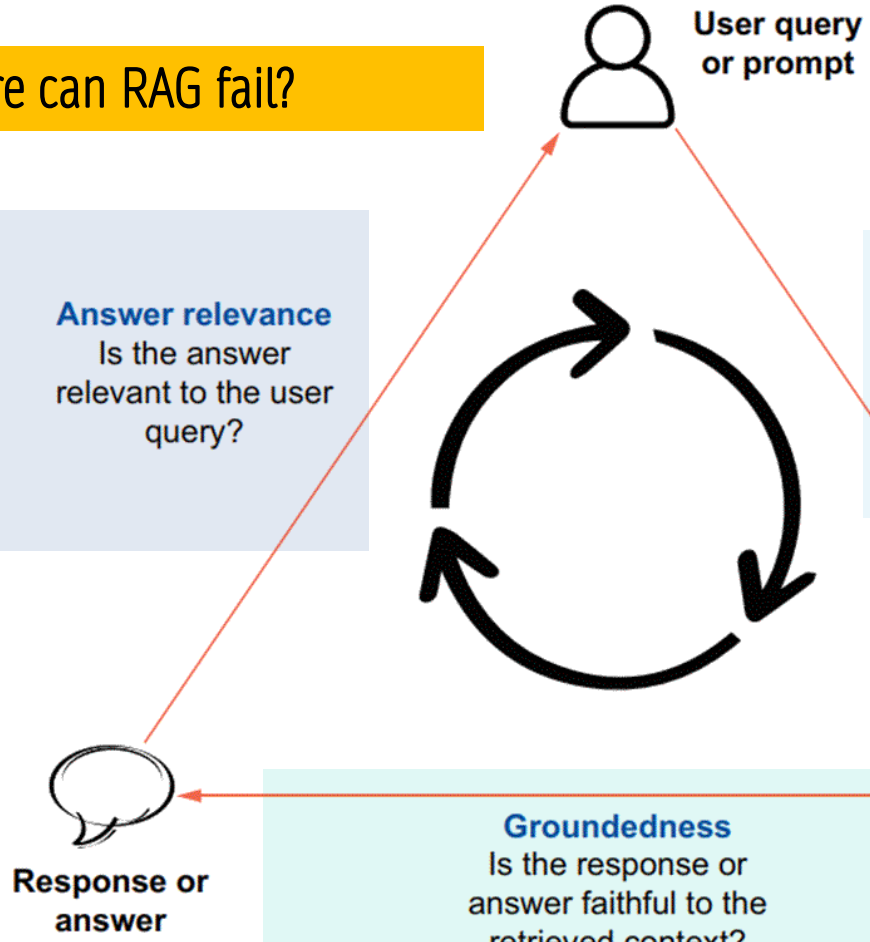


# EVALUATION

## ACCURACY, RELEVANCE, AND FAITHFULNESS OF RAG SYSTEMS

# EVALUATION

## Where can RAG fail?




**Response failure:**
The answer may be supported by the sources but still not answer the actual question.

**Retrieval failure:**
The correct evidence was not retrieved.

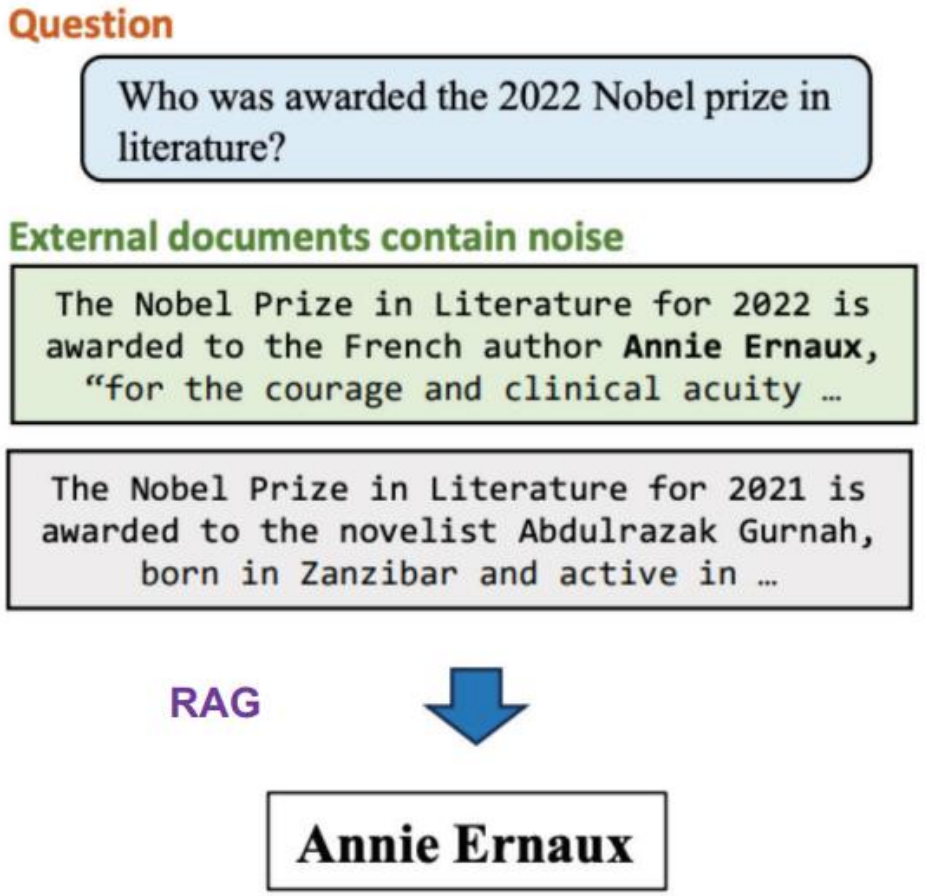
**Knowledgebase quality:**
Is the retrieved information itself correct and up to date?

**Generation failure:**
The correct evidence was retrieved, but the LLM ignored, distorted, or invented information.

# EVALUATION NOISE ROBUSTNESS

**Noise robustness**

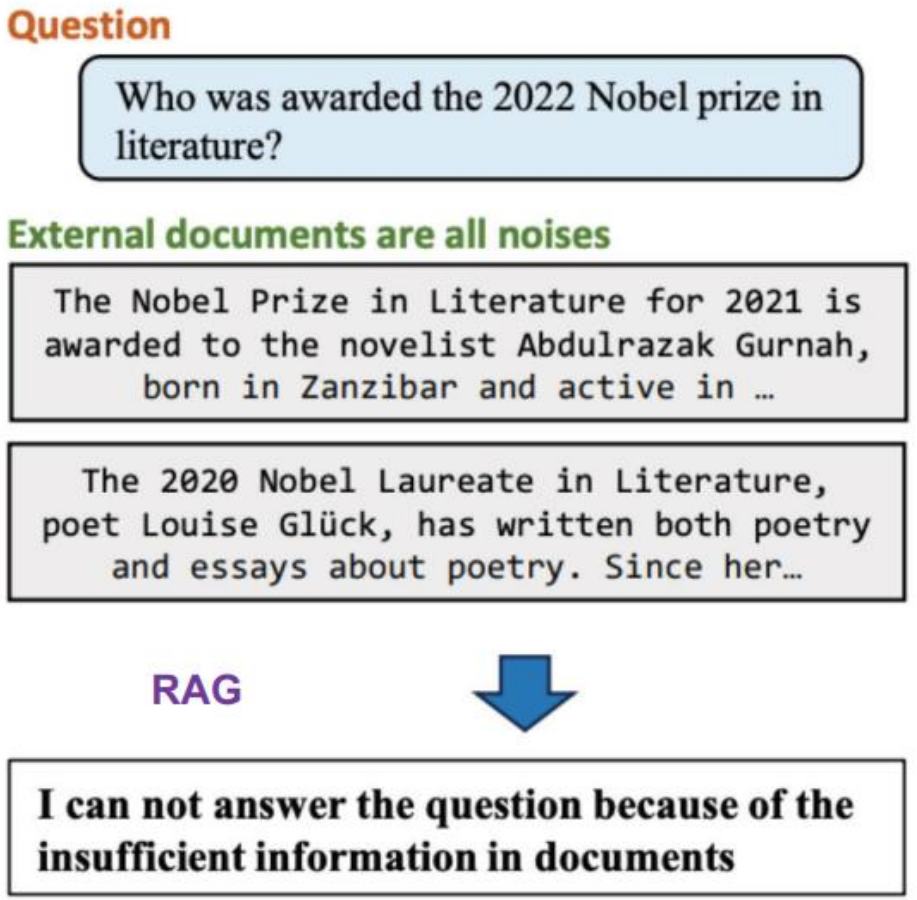
- **information** in knowledge base **not** perfectly **fitted** to every potential question
- relevance but **no** meaningful **information** to user query
- **ability to separate** noisy documents from relevant ones



# EVALUATION NEGATIVE REJECTION

**Negative rejection**

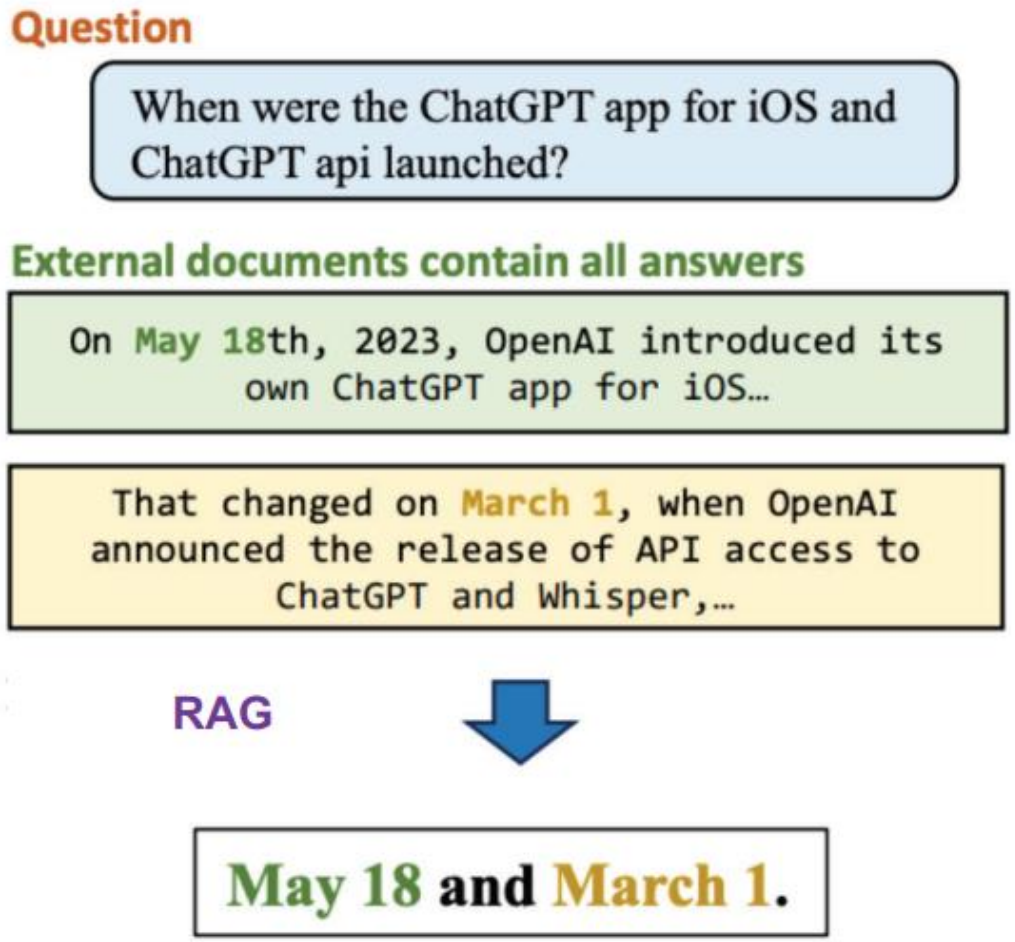
- Ability to **not give** an **answer** when there is no relevant information in knowledge base



# EVALUATION INFORMATION INTEGRATION

**Information integration**

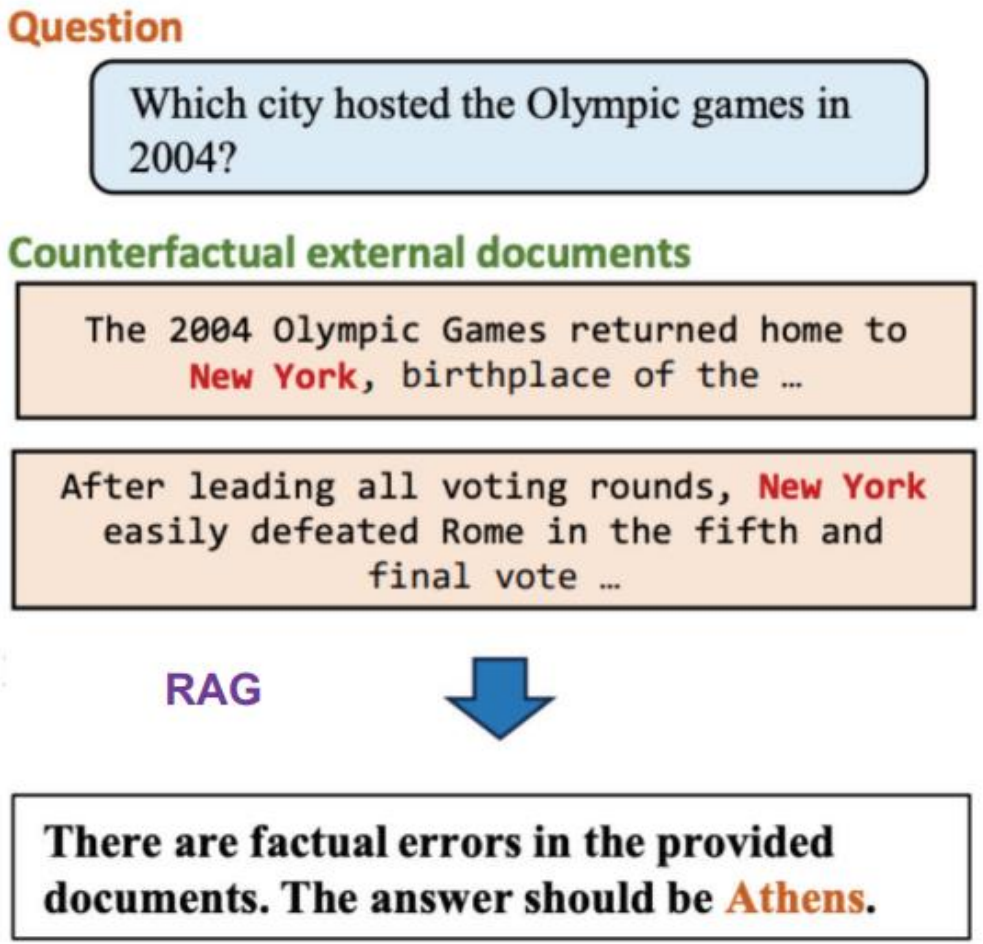
- ability of the system to **assimilate information** from multiple documents



# EVALUATION COUNTERFACTUAL ROBUSTNESS

**Counterfactual robustness**

- Information in the knowledge base might itself be **inaccurate**
- **Ability to reject** known inaccuracies in the retrieved information



# SUMMARY AND LIMITATIONS

**RAG is a powerful tool for integrating "facts" into LLM's**

It consists of

- **indexing** pipeline for **knowledgebase** creation
- **generation** pipeline for generating **contextual responses**

## Limitations

**Latency:** Indexing (offline) and generation (online) require additional computational steps.

**Retrieval quality:** Depends on strategy and hyperparameter tuning (chunking, embedding, …).

**Limited reasoning:** Difficulty in integrating retrieved information from several documents.

**Knowledgebase quality**: RAG can faithfully use retrieved information that is itself wrong, outdated or malicious.